RockyBorg minimum project requirements

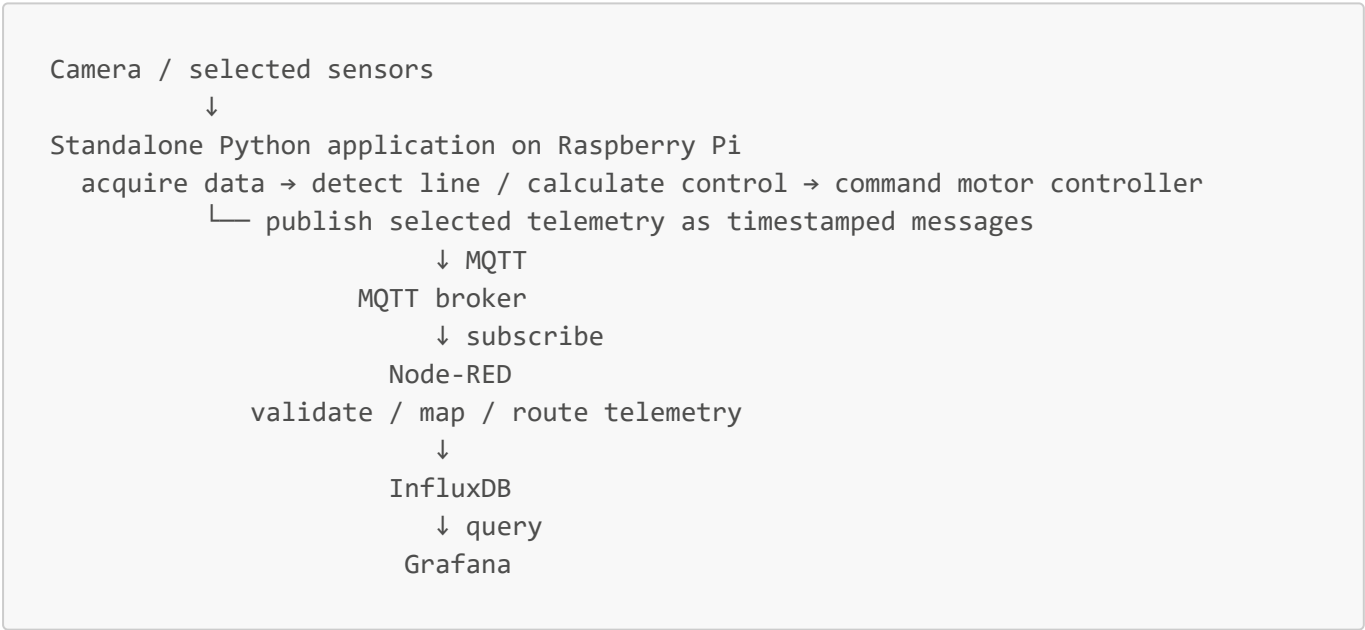
Status: Planning draft for team review
Scope: Standalone Python 3 with MQTT, a broker, Node-RED, InfluxDB, and Grafana. ROS 2 is outside this draft.

Purpose and scope

This document turns the milestones in Meetings and Tasks into a minimum, evidence-based requirement list. The project-specific Python vs. ROS 2 comparison and Python telemetry flow diagram provide supporting context.

The Python and telemetry stack below is the scope requested for this draft. It is not a record that the whole team has formally selected that stack. The meeting notes contain candidate milestones from basic control through mapping; they do not say which later milestones are assessed. This draft therefore identifies a small demonstrator as the minimum baseline and marks advanced milestones conditional. Confirm the course rubric and acceptance thresholds with the team or instructor.

Proposed system flow



Python owns robot sensing and real-time movement. MQTT and the broker carry telemetry to Node-RED. Node-RED prepares and routes it to InfluxDB; Grafana presents stored measurements. The dashboard path is for monitoring and analysis. A dashboard or broker outage must not be allowed to leave motor behavior uncontrolled; the robot needs a locally defined safe stop/fault response. Exact behavior remains to be agreed and tested with the actual motor controller.

Minimum requirements by milestone

Milestone	Minimum project requirement	Evidence that demonstrates it	Scope
1. Hardware and basic control	Inventory the actual Raspberry Pi, camera, sensors, motors, and motor controller. On the selected Pi operating system, start the Python application, read at least one chosen line-detection input, and issue a controlled command through the actual motor controller. Define a safe start/stop and fault response before moving the robot.	Hardware/interface inventory; repeatable camera or light-sensor read; motor-controller smoke-check record; observed stop/fault behavior; setup instructions another teammate can follow.	**Baseline**
2. Straight-line following	Use the camera or light sensor to detect a straight line, calculate a steering/control correction in Python, and make the robot follow a defined straight test path. Keep the control loop local to the robot so telemetry services are not required for movement.	Test setup and run record; sensor/line observations and steering output; successful straight-path run; failures and tuning noted. Agree path dimensions and pass criteria before assessment.	**Baseline demonstrator**
3. Curved-path control and telemetry	Extend the controller to a defined curved path and tune it. Publish selected time-series telemetry from Python over MQTT; receive it in Node-RED; write it to InfluxDB; and display/query it in Grafana.	Curved-path run; a trace showing timestamped robot data arriving through each stage; Grafana panel for the same run; recorded controller settings and observations.	**Next milestone; confirm whether required for minimum course demonstration**
4. Sensor streaming and fusion	Stream measurements from the chosen available sensors. Compare individual readings with a fused estimate or control result, and document whether fusion improves the chosen task. Do not claim sensor fusion merely because several sensor streams are displayed.	Sensor data with units/timestamps; documented fusion method; comparison against single-sensor baseline; conclusion with limitations.	**Conditional on scope, hardware, and course requirements**
5. Object detection / environmental awareness	If required, detect a defined object or obstacle and specify what robot response should follow.	Repeatable test cases, detection results, and observed response; defined limits and failure cases.	**Conditional; not required for the minimum baseline**

Milestone	Minimum project requirement	Evidence that demonstrates it	Scope
6. Localization, SLAM, or mapping	If required, select a localization/mapping objective and suitable sensors, then evaluate the resulting position/map against a stated reference or test.	Recorded run, generated position/map output, reference comparison, and limitations.	**Conditional; not required for the minimum baseline**

Minimum software and telemetry requirements

Area	Requirement
Python robot application	Separate hardware access, perception/line detection, control, and communication into understandable modules. Keep motor decisions local. The current `project/src` files are empty placeholders, so these capabilities are not implemented yet.
MQTT and broker	Select and configure a broker reachable by the Python publisher and Node-RED subscriber. Define topic names, message format, timestamp convention, units, and behavior on disconnect. Keep credentials/configuration out of source control. Broker product, hosting location, and security settings are open choices.
Node-RED	Subscribe to the agreed MQTT telemetry topics; validate/parse incoming messages; map fields to the agreed measurement schema; handle malformed or missing data visibly; and send valid measurements to InfluxDB.
InfluxDB	Persist the selected time-series telemetry with a documented measurement/field/tag schema and timestamps so runs can be queried later. Choose the InfluxDB version and matching Node-RED integration before implementation.
Grafana	Connect to the project InfluxDB data source and show at least one useful panel for a test run. Identify units, time range, and the selected robot/run where applicable.
End-to-end observability	Demonstrate at least one measurement produced by Python appearing in InfluxDB and Grafana through the configured MQTT and Node-RED path. Record enough setup detail for a teammate to reproduce it.
Separation of control and monitoring	The robot must continue to have a defined safe local control behavior if the broker, network, Node-RED, database, or dashboard is unavailable. The remote monitoring stack is not a substitute for motor safety.

Out of the minimum baseline unless required

- ROS 2 nodes, topics, packages, and launch system.
- Gazebo simulation.
- Curved-path following, sensor fusion, object detection, localization, SLAM, and 3D mapping until their status is confirmed against course requirements.
- QR-marker navigation, homography/camera-pose estimation, depth cameras, or LiDAR unless selected to meet a confirmed milestone.

- Sending full camera frames to Node-RED, InfluxDB, or Grafana. The initial telemetry path should carry selected measurements and control/status data; camera image processing stays in Python unless the team later defines another need.

Decisions and acceptance criteria still to confirm

- Official course deliverables, which milestones are assessed, and what evidence counts as passing.
- Exact Raspberry Pi operating system and Python/runtime versions.
- Actual motor, motor controller, steering mechanism, and supported interface.
- Which sensor is used for the first line-following demonstration and the test path/lighting conditions.
- MQTT broker product and deployment location; network, authentication, and reconnect behavior.
- InfluxDB version, Node-RED nodes/integration, retention needs, and Grafana deployment location.
- Telemetry topic names, schema, sampling rates, units, timestamps, and required dashboard panels.
- Quantitative acceptance thresholds such as path completion, control-loop rate, or telemetry delay. None are specified in the available project notes, so this document does not invent them.

Current implementation status

This is a requirements draft, not proof of implementation. The current `project/src` files are empty placeholders. Hardware availability, the motor/controller interface, deployment versions, test conditions, and course acceptance criteria still need confirmation. Record implementation evidence as each requirement is completed.